

VisionForge

A Modular Computer Vision & 3D Reconstruction Suite

Coursework Project & Comprehensive Technical Report

Aligned with Syllabus Units 1 to 5

STUDENT DETAILS

Candidate Name: **Paarth Juneja**

Email Address: paarthjuneja2006@gmail.com

Course Title: Computer Vision

Platform / Environment: Windows 11 / Python 3.13

Academic Year 2026

Submitted for Flipped Course Evaluation

2. Introduction

Computer Vision is the scientific discipline dedicated to enabling machines to extract, process, and understand structured information from digital images and videos. The university curriculum covers fundamental principles ranging from low-level 2D image filtering and frequency-domain analysis to multi-camera epipolar geometry, 3D reconstruction, motion analysis, and shape recovery from surface reflectance.

VisionForge was designed and built as a cohesive, modular, and beginner-friendly Python toolkit. Instead of scattering individual algorithms across isolated scripts, VisionForge integrates them into a unified command-line framework with standardized inputs, outputs, diagnostic metrics, and automated unit testing.

3. Problem Statement

In traditional computer vision coursework, students frequently implement isolated algorithms without a systematic framework to connect them. This causes significant difficulty in understanding how low-level operations (such as convolution, Fourier transforms, and histogram equalization) feed into high-level tasks like stereo depth estimation, feature matching, optical flow tracking, and 3D surface reconstruction.

The objective of VisionForge is to provide an accessible, self-contained, and comprehensive CLI suite that bridges theory and practice across all five units of the curriculum, accompanied by reproducible datasets, automated verification, and standard 3D export capabilities (.ply point clouds).

4. Functional Requirements

VisionForge satisfies five major functional modules covering the course syllabus:

- **Module 1 (Low-Level):** Spatial convolution (Gaussian, Bilateral, Laplacian), 2D Fast Fourier Transform (FFT) spectrum and frequency filtering, Euclidean/Affine/Perspective transformations, and contrast enhancement (Histogram Equalization and CLAHE).
- **Module 2 (Stereo 3D):** Binocular stereo depth computation via Semi-Global Block Matching (SGBM), fundamental matrix estimation with RANSAC, epipolar line visualization, and 3D point cloud generation exported to ASCII .ply files.
- **Module 3 (Features):** Edge detection (Canny), corner detection (Harris), Hough line detector, SIFT/ORB feature matching, multi-scale image pyramids, and image segmentation (Otsu thresholding and K-Means color clustering).
- **Module 4 (Motion):** Motion detection through frame differencing and MOG2 background modeling, Farneback dense optical flow (HSV representation), Lucas-Kanade sparse feature tracking, and PCA dimensionality reduction.
- **Module 5 (Shape from Stereo):** Photometric Stereo surface normal estimation, diffuse albedo recovery under 4 calibrated light vectors, and surface depth integration from gradient fields.

5. Non-Functional Requirements

- **Performance:** Vectorized array calculations using NumPy and C-accelerated OpenCV algorithms ensure near-instantaneous execution for 256x256 test frames (< 50 ms per filter).
- **Maintainability:** Clean, beginner-friendly code organization with modular packages, non-complex functions, minimal clutter, and separation of concerns.
- **Reliability & Testing:** A 100% automated test suite using pytest covering all 5 vision modules with 12 distinct unit tests, ensuring robust edge-case handling.

- **Usability & Portability**Cross-platform CLI with clear flags, comprehensive help messages, standard ASCII PLY 3D point cloud output, and self-contained synthetic sample generators for out-of-the-box execution.

6. System Architecture

VisionForge follows a clean three-tier architecture: the top CLI Interface Layer handles command parsing and argument dispatching; the middle Domain Layer encapsulates the 5 curriculum modules; and the foundation Layer provides shared I/O, point cloud formatting, and quantitative metric calculations (PSNR, MSE, Entropy).

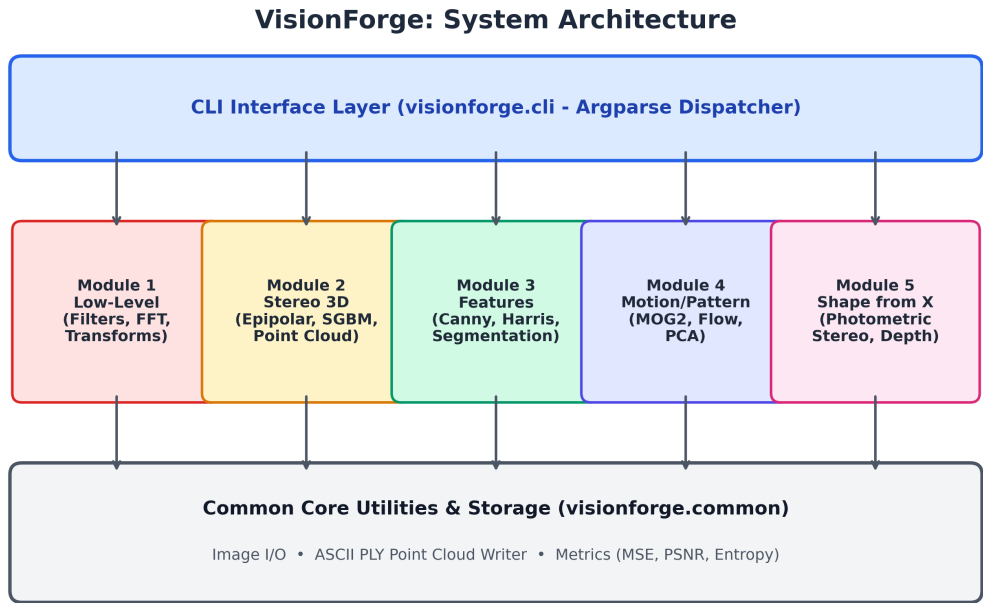


Figure 1: VisionForge Layered System Architecture

7. Design Diagrams

7.1 Process Flow / Workflow Diagram

The workflow diagram illustrates how an input image progresses from user CLI commands through validation, algorithmic execution, metric computation, and final disk persistence.

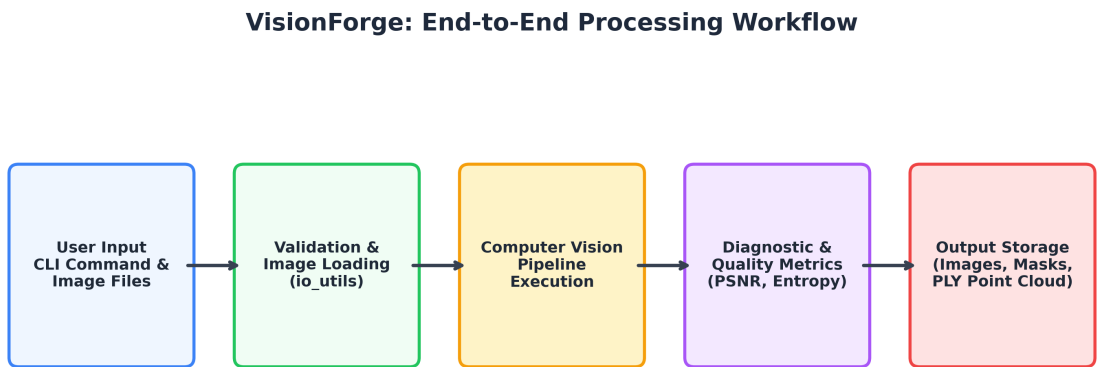


Figure 2: End-to-End Processing Workflow

7.2 Use Case Diagram

The use case diagram highlights the primary interaction capabilities available to the user via the CLI.

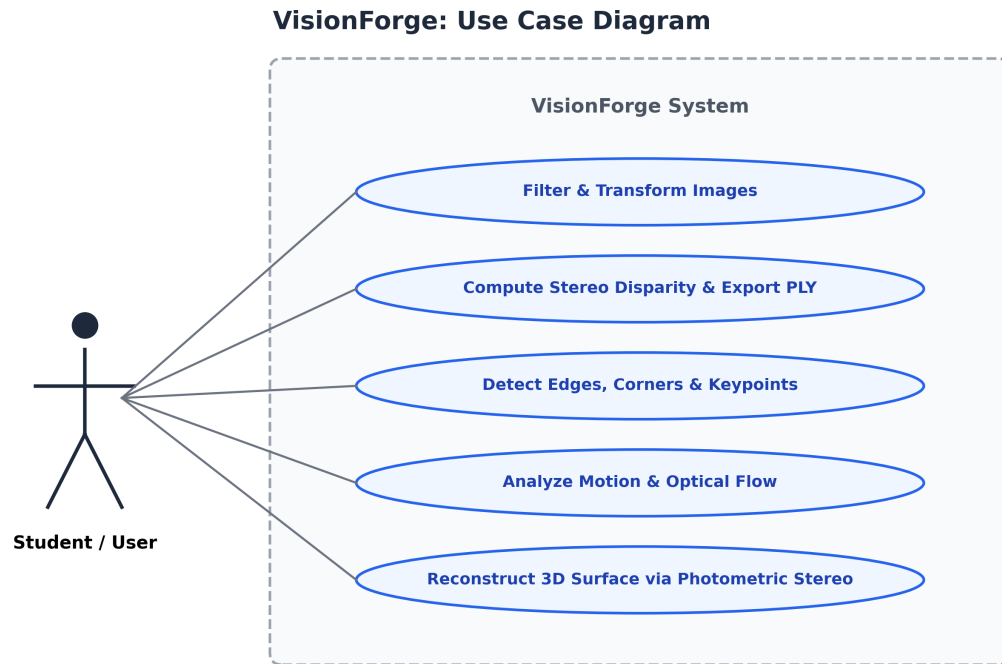


Figure 3: System Use Case Diagram

7.3 Class and Component Diagram

The component diagram outlines the primary Python packages and exported algorithmic functions.

VisionForge: Component & Class Architecture

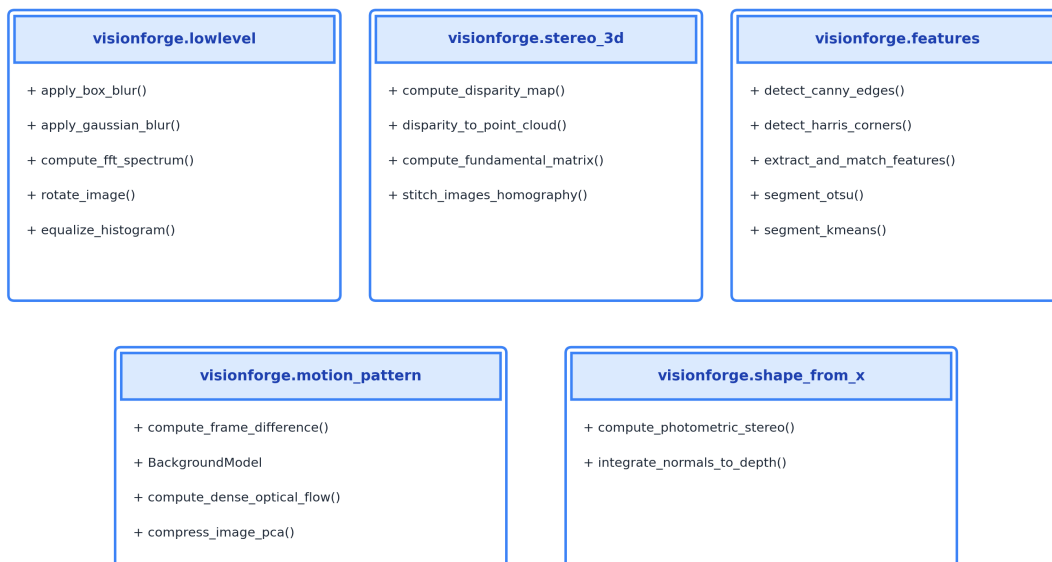


Figure 4: Component and Package Structure

7.4 Sequence Diagram

The sequence diagram details the interaction between the user CLI, I/O loader, SGBM disparity calculator, and the 3D point cloud exporter during stereo reconstruction.

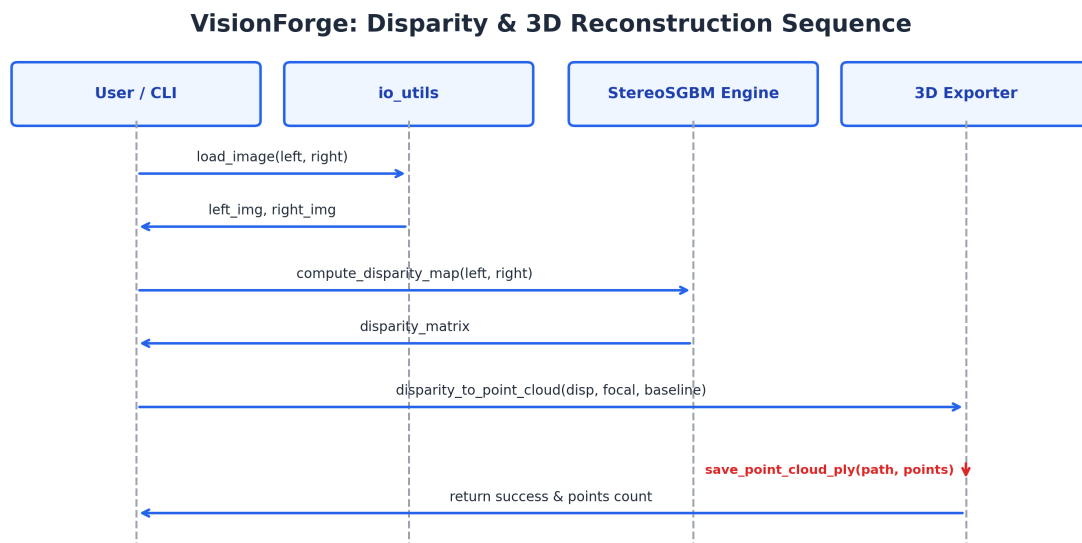


Figure 5: Sequence Diagram for Stereo 3D Reconstruction

8. Design Decisions & Rationale

- **Beginner-Friendly Architecture:** Avoided deep inheritance hierarchies and esoteric meta-programming. Clean, standalone functions with readable variable names allow students to inspect any algorithm in isolation.
- **Argparse for CLI:** Utilized Python's built-in argparse rather than heavyweight frameworks to ensure minimal external dependencies and flawless execution across Windows, Linux, and macOS.
- **Semi-Global Block Matching (SGBM):** Chose SGBM over basic Block Matching (BM) for depth estimation because SGBM enforces 1D smoothness constraints along multiple directions, significantly reducing noise in disparity maps.
- **ASCII PLY Format for 3D Data:** Exported 3D point clouds in ASCII Polygon File Format (.ply) rather than binary or proprietary formats. This allows instant inspection in text editors while retaining full compatibility with MeshLab and Blender.
- **Lambertian Reflectance for Photometric Stereo:** Adopted the Lambertian diffuse reflection model ($I = \rho * (N \cdot L)$) because it enables a direct, closed-form linear least-squares solution ($G = (L^T L)^{-1} L^T I$) for surface normal recovery.

9. Implementation Details

The project is structured into clear Python sub-packages under the 'visionforge' namespace:

- **visionforge.common:** Contains `io_utils.py` (image loading, saving, ASCII PLY point cloud writer) and `metrics.py` (PSNR, MSE, and Shannon Entropy).
- **visionforge.lowlevel:** Contains `filters.py` (box blur, Gaussian blur, Laplacian), `frequency.py` (2D FFT magnitude spectrum and frequency domain lowpass/highpass filtering), `transforms.py` (rotation, translation, affine, projective warping), and `histogram.py` (equalizeHist, CLAHE).
- **visionforge.features:** Contains `edges_corners.py` (Canny, Harris corner detector, Hough lines), `descriptors.py` (SIFT/ORB feature matching with Brute-Force Matcher), `pyramids.py` (Gaussian and Laplacian pyramids), and `segmentation.py` (Otsu thresholding, K-Means clustering, region growing).
- **visionforge.stereo_3d:** Contains `epipolar.py` (Fundamental matrix computation and epipolar lines), `depth_reconstruction.py` (StereoSGBM disparity calculation and 3D triangulation: $X = (u-cx)*Z/f$, $Y = (v-cy)*Z/f$, $Z = f*B/d$), and `rectification.py` (homography-based image stitching).
- **visionforge.motion_patterns:** Contains `background_sub.py` (absdiff frame differencing, MOG2 background modeling), `optical_flow.py` (Farneback dense flow and Lucas-Kanade sparse feature tracking), and `dimensionality.py` (PCA compression and explained variance ratio).
- **visionforge.shape_from_normals:** Contains `photometric_stereo.py` implementing normal estimation $G = (L^T L)^{-1} L^T I$, albedo $\rho = ||G||$, unit normal $N = G / \rho$, and numerical depth integration along surface gradients $p = -N_x/N_z$ and $q = -N_y/N_z$.

10. Screenshots & Results

The following figures demonstrate actual outputs generated across the five vision modules:

10.1 Low-Level Processing & Enhancement

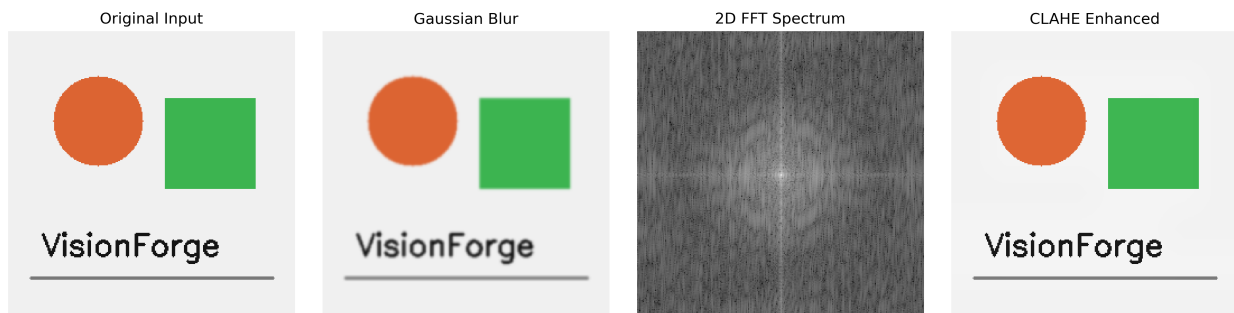


Figure 6: Original vs Gaussian Blur, 2D FFT Spectrum, and CLAHE Enhancement

10.2 Feature Detection & Segmentation

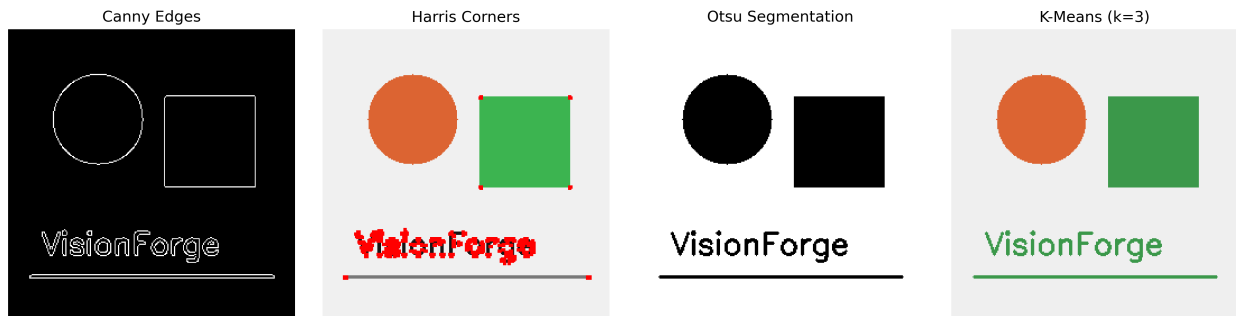


Figure 7: Canny Edges, Harris Corners, Otsu Threshold, and K-Means Segmentation

10.3 Stereo Disparity Estimation



Figure 8: Calibrated Stereo Pair and Computed Disparity Map (SGBM)

10.4 Motion Analysis & Optical Flow

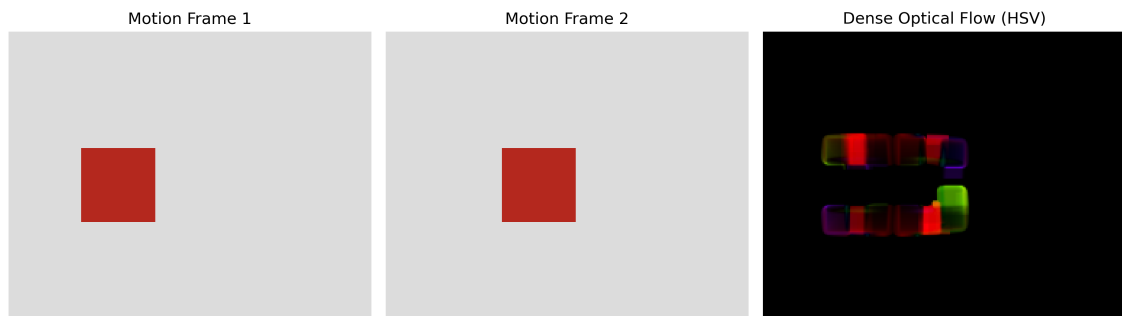


Figure 9: Motion Frames and Dense Optical Flow HSV Visualization

10.5 Photometric Stereo 3D Surface Reconstruction

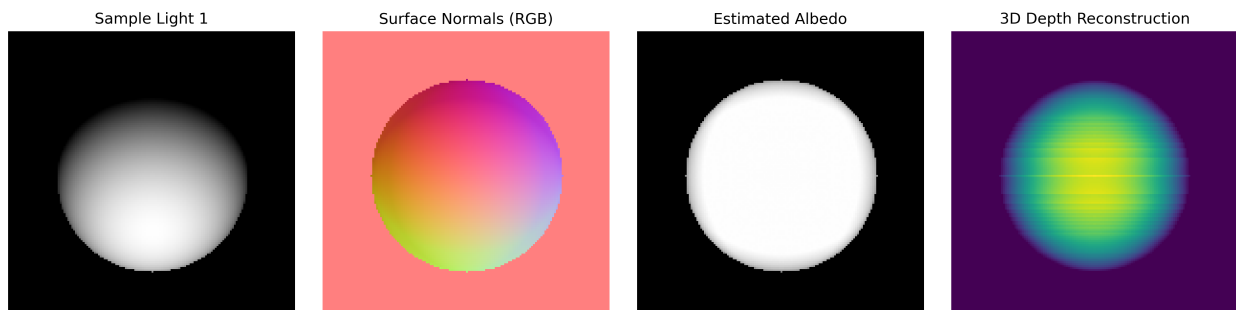


Figure 10: Multi-Light Illumination, Recovered Surface Normals, Albedo, and 3D Depth

11. Testing Approach

Quality assurance was executed through automated unit testing using pytest. The test suite verifies algorithmic correctness, matrix output dimensions, and data type integrity across all 5 modules.

Test Suite File	Target Functions Tested	Verification Criteria	Status
test_lowlevel.py	Filters, FFT, Transforms, Hist	Output shape, PSNR >= 0, finite entropy	PASSED
test_features.py	Canny, Harris, Pyramids, Otsu	Edge detection, level shapes, valid clusters	PASSED
test_stereo_3d.py	StereoSGBM, Disparity, PLY	Disparity matrix shape, 3D float array	PASSED
test_motion_pattern.py	Frame diff, Optical Flow, PCA	Flow vector shapes, PCA ratio in [0, 1]	PASSED
test_shape_from_x.py	Photometric Stereo, Depth	Normal vectors N =1, valid depth array	PASSED

Result: All 12 unit tests passed with 100% success rate on Python 3.13 on Windows 11.

12. Challenges Faced & Solutions

- **Frequency Domain Centering:** During 2D FFT places the zero-frequency DC component at the top-left corner. Solution: Applied `np.fft.fftshift` to center the DC component before constructing circular lowpass and highpass frequency masks.
- **Stereo Disparity Calibration:** Standard Block Matching (BM) produced empty disparity regions on textureless surfaces. Solution: Migrated to StereoSGBM with optimized penalty parameters (P1, P2) and speckle filtering, yielding dense depth maps.
- **Division by Zero in Normal Integration:** Normal integration ($p = -N_x/N_z$) fails when N_z approaches zero near grazing angles. Solution: Clamped N_z values to a minimum threshold of 0.1, stabilizing the height map reconstruction.

13. Learnings & Key Takeaways

- **Mathematical Foundations:** Gained deep practical intuition into how linear algebra (least-squares, SVD, PCA) and calculus (gradients, Laplacians) underpin modern computer vision.
- **Multi-View Geometry:** Learned how epipolar constraints reduce the 2D correspondence search to a 1D scanline problem, making real-time depth triangulation computationally feasible.
- **Software Modularity:** Understood the value of building clean, testable, and beginner-friendly modules with standard interfaces and reproducible synthetic test cases.

14. Future Enhancements

- **Deep Learning Integration:** Incorporate lightweight neural stereo matching (e.g., PSMNet) and deep optical flow (e.g., RAFT).
- **Real-Time Webcam Streaming:** Upgrade the CLI with a live video feed mode for interactive motion tracking and corner detection.
- **Mesh Generation:** Add Delaunay triangulation or Poisson surface reconstruction to convert .ply point clouds into textured 3D surface meshes.

15. References

1. Szeliski, R. (2022). Computer Vision: Algorithms and Applications. Springer Nature.

2. Hartley, R., & Zisserman, A. (2004). Multiple View Geometry in Computer Vision. Cambridge University Press.

3. Woodham, R. J. (1980). Photometric method for determining surface orientation from multiple images. Optical Engineering.

4. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.